

What is Terminal and Shell ?

terminal : A CLI (Command Line Interface) application. (like browsers but for command-based interaction).

shells : The programs that runs inside the terminal (like tabs). e.g., bash shell, windows powershell, command prompt, Zsh shell

bash : scripting language like programming language.

BASIC SHELL COMMANDS

1. **echo** : used for print output. e.g., echo Hello world.
 - `echo $name` => using variable.
 - `echo $((num1 + num2))` => performing operation on variable
2. **pwd** : present working directory.
3. **whoami** : name of current user name
4. **ls** (list directory/folder) : lists contents of a directory.
 - `ls -a` => lists all contents including hidden files.
 - `ls -la` => shows detailed info (permissions, size, etc.)
 - `ls -l` => permissions
5. **cd** (change directory) : used to change directory.
 - `cd path/directoryName`
 - `cd ./` => refer current directory
 - `cd ../` => one step back. (refer parent directory)
 - `cd ~/` => home directory/user
 - `cd /` => root directory.

COMMANDS FOR FILE/IO

1. **touch** : Used to create files.
 - `touch filename` => `touch script.js` (create a script.js file).
 - Can be create multiple files. e.g., `touch index.html style.css script.js`
2. **mkdir** : Creates directory/folder(s).
 - `mkdir directoryName` => `mkdir programs` (create a programs folder)
 - Can be create multiple directory. e.g., `mkdir folder1 folder2`.
3. **cp** : Used copy-paste file .
 - `cp source-path(file) target-path(directory)` => e.g., `cp index.html test/` (you can also give full path if it doesn't exist in current working directory or folder).
4. **mv** : Used to move file or rename file. (both could be done simultaneously).

- `mv source-path(file) target-path(directory)` => Moved.
- `mv source-path(file/directory) target-path(file/directory)` => Renamed (if same source and target).
- `mv style.css testDirectory` (moving).
- `mv style.css design.css` (renaming).
- You can also give full path if it doesn't exist in current working directory or folder

5. **rm** : Used to remove/delete file.

- `rm filename` => e.g., `rm script.js`
- `rm -r` : Removes non-empty directory => `rm -r directoryName/folderName`

6. **rmdir** : Used to remove empty directory/folder.

- `rmdir directoryName/folderName` => e.g., `rmdir folderName`

COMMANDS FOR READING FILE AND EDITING

1. **cat** : Used to read file.

- `cat filename`. e.g., `cat script.js` (read script.js).**

2. **nano** : Basic user friendly file editor.

- `nano filename`. e.g, `nano script.js` (open script.js in nano editor).
- `Ctrl + S` : Saved file.
- `Ctrl + X` : Exit from the editor

3. **vim** : Very powerful file editor.

- `vim filename`. e.g., `vim script.js` (open script.js in vim editor).
- Two Modes : **Normal** (press `Esc` = write comand) mode and **Insert** (press `i` or when you type) mode.
- `:w` => Write or save.
- `:q` => Exit from the editor.
- `:q!` => Exit from the editor without saving or writing the changes.
- `:wq` => Save and exit from the file.

HOW TO CREATE ALIASES FOR COMAND SHORTCUTS

- Create `.bashrc` file in home directory.
- e.g., `alias aliasName = "comandName"` (inside `.bashrc` file).

.bashrc

What is `.bashrc`?

`.bashrc` is a script that runs **every time a new terminal session** starts in interactive **Bash shell** (on Linux/Unix systems).

Key Uses:

- Set environment variables
- Customize the command prompt (**PS1**)
- Create aliases for commands
- Run commands automatically at terminal startup

Common Examples:

```
# Alias example
alias ll='ls -la'

# Custom prompt
PS1="\u@\h:\w$ "

# Add custom PATH
export PATH="$PATH:/home/user/scripts"
```

Location:

- File path: `~/.bashrc`
- Only affects the **current user**

How to Apply Changes:

After editing:

```
source ~/.bashrc
```

Tip:

- `.bashrc` runs **only in interactive non-login shells**
- For login shells, use `.bash_profile` or `.profile`

Bonus Tip:

Use `.bashrc` to auto-activate virtual environments or show welcome messages:

```
echo "Welcome, Afnur!"
```